Future Generation Computer Systems 155 (2024) 367–383

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

A multi-objective approach for communication reduction in federated learning under devices heterogeneity constraints

José Ángel Morell [b,*], Zakaria Abdelmoiz Dahi [a,b], Francisco Chicano [b], Gabriel Luque [b], Enrique Alba [b]

[a] Univ. Lille, Inria, CNRS, Centrale Lille, UMR 9189 CRIStAL, F-59000 Lille, France
[b] ITIS Software, University of Malaga, Spain

ARTICLE INFO

Keywords:
Communication cost
Federated learning
Genetic algorithm
Model compression
Multi-objective optimisation
Neural network

ABSTRACT

Federated learning is a paradigm that proposes protecting data privacy by sharing local models instead of raw data during each iteration of model training. However, these models can be large, with many parameters, provoking a substantial communication cost and having a notable environmental impact. Reducing communication overhead is paramount but conflictual to maintaining the model's accuracy. Most research has dealt with the different factors influencing communication reduction separately without addressing their correlations. Moreover, most of them do not consider the heterogeneity of clients' hardware. Finding the optimal configuration to fulfil all these training aspects can become intractable for classical techniques. This work explores the add-in that multi-objective evolutionary algorithms can provide for solving the communication overhead problem while achieving high accuracy. We do this by 1) realistically modelling and formulating this task as a multi-objective problem by considering the devices' heterogeneity, 2) including all the communication-triggering aspects, and 3) applying a multi-objective evolutionary algorithm with an intensification operator to solve the problem. A simulated client–server architecture of four devices with four different processing speeds is studied. Both fully connected and convolutional neural network models are investigated with 33,400 and 887,530 weights, respectively. The experiments are performed using the MNIST and Fashion-MNIST datasets. A comparison is made between three approaches using an extensive set of metrics. Results prove that our approach obtains solutions with better accuracy than the full-communication setting and other methods while getting reductions in communications by around 1,000 times in most cases and up to 10,000 times in some cases compared to the maximum communication setting.

1. Introduction

Machine Learning (ML) [1] is one of the most active axes in Artificial Intelligence (AI). Data abundance is one of the main factors that lead to its flourishing. However, the number of edge devices [2] (nearer the users), such as laptops, smartphones, self-driving cars, drones, robots, smart lights, smart refrigerators, wearable devices, and more, is increasing. The amount of data they generate at the edge is growing exponentially [3,4], which prevents us from storing/processing all this data in the cloud. Also, data privacy is a current hot topic, especially in many ML application domains (e.g., healthcare). Other devices need to make decisions in real-time and cannot wait for the cloud to answer them before taking action, or they can even be temporarily uncommunicated [5,6]. As a promising solution to these issues, Federated Learning (FL) [7–9] leverages the calculation power of distributed devices (i.e., clients) that cooperatively work on learning a shared global model while maintaining data locally and private. The devices train a local model and then send its parameters instead of the data to the server, which aggregates them to learn a shared global model. FL proved to be efficient and widely applied [10–12]. However, these models may have a significant size, containing numerous parameters, resulting in a considerable communication cost and a severe environmental impact.

Client–server communication is at the core of the FL paradigm. The original Federated Averaging (FedAvg) algorithm [8] trains an artificial neural network (ANN) model for a certain number of local steps on each device and subsequently performs a round of communication. Therefore, finding the correct number of communication rounds to reduce the data sent while maintaining a high accuracy is a challenge

* Corresponding author.
E-mail addresses: jamorell@uma.es (J.Á. Morell), abdelmoiz-zakaria.dahi@inria.fr (Z.A. Dahi), chicano@uma.es (F. Chicano), gluque@uma.es (G. Luque), ealbat@uma.es (E. Alba).

https://doi.org/10.1016/j.future.2024.02.022
Received 28 April 2023; Received in revised form 20 February 2024; Accepted 23 February 2024
Available online 24 February 2024

considering their conflicting relationship. Previous literature has already researched some overhead-provoking factors (e.g., the number of communication rounds, size of the models, etc.) [13–15] but separately and analyses on their correlation and applicability together have rarely been conducted. Furthermore, they often experiment with idealised FL configurations in which learning occurs using look-like rather than heterogeneous [16–19] clients (e.g., laptops, smartphones, etc.), which is poorly representative of real-world scenarios. Other authors have proposed a multi-objective approach in which they evolve the ANN architecture [20]. This work instead investigates, within the same study, the main parameters triggering communication overhead that the literature usually tackles separately.

Reducing the communication in FL stands in finding the optimal hardware (e.g., number of participating clients) and software (e.g., the models' size, training steps, etc.) configurations. Doing so is a complex task where classical manual or exhaustive methods are useless. In this context, Evolutionary Algorithms (EA) have proved to be a promising alternative [21]. With this in mind, this paper investigates the contribution of EA to solving the Communication Reduction Problem (CRP) in a realistic FL environment. That is done by (1) modelling and formulating the CRP as a realistically multi-objective problem that considers principal factors inducing such issues and the heterogeneity of the devices, (2) including all the communication-triggering aspects, and (3) applying a multi-objective evolutionary algorithm (NSGA-II) [22] coupled with an Estimation Distribution-based Algorithm (EDA) [23] to solve the problem. The proposal has been assessed using a client–server architecture of four devices with four different processing speeds. A fully connected and a convolutional ANN model are investigated, with 887,530 and 33,400 weights, respectively. The `MNIST` and `Fashion-MNIST` datasets, both of 70,000 handwritten digits and clothing images, respectively, have been used. A comparison has been made against three state-of-the-art approaches using an extensive set of metrics.

This work is a continuation of an earlier work [24] in which the communication overhead problem in FL using homogeneous devices was modelled as a multi-objective optimisation problem, and a multi-objective optimisation algorithm (NSGA-II) was applied to solve it, aiming to minimise the communicated data while maintaining or even improving accuracy. All the devices ran the same number of local steps, which are a local weights update using a mini-batch of data. The variables to optimise were the number of devices participating in each communication round, the number of local steps, and the amount of quantisation and sparsification used in each ANN layer.

However, the edge devices are inherently heterogeneous, requiring a more in-depth exploration of addressing this heterogeneity most efficiently. Employing homogeneous devices or approaching this issue from such a perspective is simpler but unrealistic. This study represents a significant improvement over the previous work. We model the CRP considering the heterogeneity of devices and their variance in the number of local steps performed between two communication rounds. We also hypothesise that the NSGA-II algorithm could benefit from an intensification operator for enhancing specific solutions. In other words, solutions on the Pareto front with lower accuracy despite minimal communication are of limited interest to us. Given our goal of obtaining solutions with the highest possible accuracy while minimising communication, we propose incorporating an intensification operator for solutions on the Pareto front with superior accuracy. This new technique is a hybrid in which an EDA [23] is incorporated into the NSGA-II to enhance its performance. In particular, we use the Univariate Marginal Distribution Algorithm (UMDA) [25]. A new dataset is studied, Fashion-MNIST and the optimisation scope is expanded by increasing the number of parameters to be optimised. That includes considerations of various aggregation techniques based on the number of local steps performed by each device, decisions on whether to share accumulated gradients or ANN weights, different techniques to handle the weights not sent during sparsification, and an analysis of the impact of variations in the learning rate and batch size. In addition, we have aimed to delve more deeply into the correlations among the model parameters in the final solutions, providing the reader with significantly more detailed insights compared to our previous work.

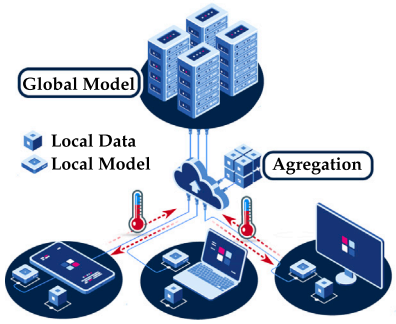


Fig. 1. Workflow of federated learning.

In summary, the main significant contributions of this work are:

- We model the CRP as a multi-objective optimisation problem, accounting for device heterogeneity and varied steps between communication rounds. Our dual objectives are to achieve high accuracy while minimising communication overhead.
- We introduce an innovative integration of NSGA-II with an EDA to tackle the CRP, resulting in enhanced accuracy compared to existing methods.
- We analyse the correlations and distributions of the final parameters within our proposed solutions. This analysis reveals a consistent pattern of getting the best non-dominated solutions when using extensive quantisation and sparsification efforts in layers with more weights while maintaining intermediate values in other layers.

The remainder of the paper is structured as follows. Section 2 introduces fundamental concepts of FL, the communication reduction techniques, and the devices' heterogeneity. Later, Section 3 presents both the CRP modelling and the designed solver, while Section 4 is dedicated to the experimental study. Finally, Section 5 concludes the paper.

2. Background

This section presents some fundamental concepts related to federated learning and its inherent device heterogeneity.

2.1. The federated learning paradigm

Federated learning comes originally from Distributed Deep Learning (DDL) [26]. In this paradigm, a shared model on a central node (i.e., server) is collaboratively trained using local models in edge nodes (i.e., clients). Both clients and the server exchange only their model weights while maintaining the integrity and privacy of the users' data (see Fig. 1).

The FedAvg algorithm [8,27] (see Algorithm 1) illustrates the FL working mechanism using a set S of clients, where $|S| = N$, and S_t is a subset of $C \cdot N$ selected clients at iteration $t = 1, \ldots, T$ where T the maximum number of global aggregations allowed. $C \in [0, 1]$ is the fraction of clients that perform computation on each communication round, η is the learning rate, and P^k is a weight assigned to each device used for weighted aggregation. Initially, the FedAvg generates a global model w_0^* on the server. Then, it randomly chooses m clients, S_t, to participate in the training process, where m is the maximum between $C \cdot N$ and 1. Each selected client downloads the currently updated weights from the server w_t^* and trains using the local dataset for E local

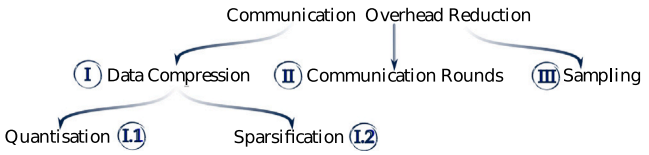


Fig. 2. Communication reduction approaches in FL.

steps. Finally, the local model weights of the S_t participating devices are sent to the server to update the global model via the aggregator, usually averaging. The whole process is executed repeatedly during T global aggregations.

Algorithm 1 FedAvg algorithm

```
Initialise(w*_0);
for t = 1, ..., T do
    m ← max(C · N, 1);
    S_t ← random_pick(S, m);
    for all clients k ∈ 1, ..., |S_t| in parallel do
        w^k_{t_0} ← w*_t
        for e ∈ 1, ..., E do
            w^k_{t_e} ← w^k_{t_{e-1}} − η∇F(w^k_{t_{e-1}});
        end for
        w^k_{(t+1)_0} ← w^k_{t_E};
    end for
    w*_{t+1} ← Σ_{k∈S_t} P^k · w^k_{t+1}/m;
end for
```

2.2. Communication reduction in distributed deep learning

Similarly to DDL, communication is also a key feature in FL. The necessary model weights exchange between clients and server generates a substantial overhead that can compromise the FL efficacy and applicability [7,9]. The literature that studies the communication reduction [28] in DDL can be classified into three categories according to the approach it uses: (I) *data compression*, (II) *decreasing communication rounds*, and (III) *reducing the number of participating devices (sampling)* (see Fig. 2).

Regarding the data compression method, two approaches can be mentioned: (I.1) quantisation [13,28] and (I.2) sparsification [14,28]. The first approach represents the data using small-sized, low-precision data (i.e., using fewer bits to represent a value). In contrast, the second approach transmits indispensable values of each communication (i.e., only some weights are transmitted). Quantisation tends to slow the convergence of the model, and the maximum data compression rate is limited to 1/32, because DDL generally uses 32-bit-coded data. Sparsification attains a 1/100 compression rate without significantly changing the model's convergence speed and accuracy. During the training process, sparsification introduces additional steps such as sampling, (de)compression, and (de)coding. These additional steps can affect the overall training efficiency, particularly in low-performance (e.g., netbook) and battery-sensitive (e.g., smartphone) devices.

Considering now the second reduction technique, decreasing communication rounds [27], the client–server communication occurs after E local steps. Training an ANN using FL requires hundreds of thousands of iterations. Thus, using sparse communication intervals allows for a reduction of the communication overhead. So, the FedAvg and its variants permit clients to execute several iterations of local training before updating the global model [27]. Concretely, the communication interval in FedAvg is controlled by the hyperparameter E. The former affects the model's accuracy-vs-training-efficiency tradeoff. From a numerical point of view, smaller values of E progress slowly but generally achieve higher accuracy. The opposite scenario improves training efficiency, but generalisation worsens by converging more quickly to a local optimum [27]. That is because when devices perform more local steps, the differences between the different devices' local weights are higher, which does not help during the aggregation process.

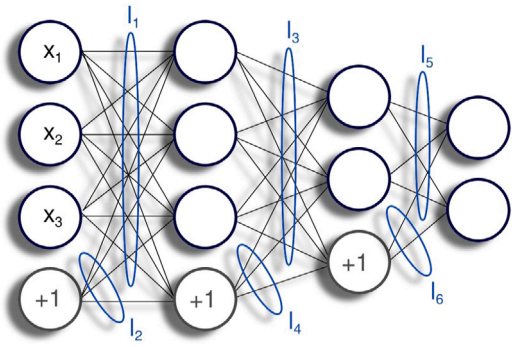


Fig. 3. An example of an ANN with six vectors of weights (i.e., $|L| = 6$).

Considering these facts, fine-tuning the FL hyperparameter E would require advanced knowledge to achieve the best efficacy.

Finally, the third technique reduces the number of participating devices (sampling), selecting a fraction of all available ones in each communication round [29,30]. The total amount of communication is reduced proportionally according to the number of selected devices.

Previous literature studies the above-presented approaches separately. However, to the best of the authors' knowledge, no work has studied all these aspects while addressing the problem of the heterogeneity of devices.

2.3. Heterogeneity in distributed learning

Classical distributed deep learning generally uses homogeneous (hardware and software) computers. However, FL uses heterogeneous hardware and software edge devices. With the deployment of 5G and 6G networks [5,31,32] and hardware development, heterogeneous edge and IoT devices have acquired higher computing and communication capabilities in a more diverse range of applications. Now, it is possible to move much of the learning we perform in the cloud to edge devices for privacy, latency, and communication efficiency. That allows us to train using a globally distributed great network of heterogeneous hardware and software devices. However, heterogeneity [17–19] adds new challenges.

Considering the unavoidable devices' heterogeneity, using the classical synchronous FL does not efficiently use the limited resources on such devices due to the waiting time required for the stragglers devices to participate in the aggregation step of each communication round. Indeed, the aggregation server and the fast devices must wait for the laggards (unreliable heterogeneous devices that could unexpectedly go offline) to update their local models. As a solution to the above-stated issues, besides the asynchronous communication [17,19,33], many researchers suggest semi-asynchronous approaches using weighted aggregation to reduce the impact of stragglers devices or stale models and improve learning efficiency [19,34]. This paper proposes a semi-asynchronous approach forcing synchrony between the slowest and fastest devices by calculating the number of steps to be performed by an ideal device with a speed of 1.0. Then, the remaining devices perform a certain number of local steps proportional to their velocity normalised to the unit velocity of the ideal device. Also, different techniques for weighted aggregation are used and studied.

3. The proposed approach

This section describes the communication reduction problem (CRP) and the proposed solver.

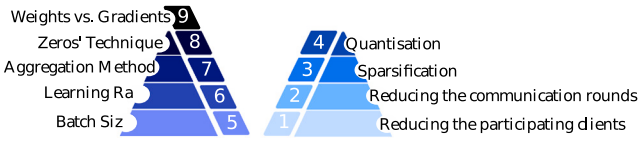


Fig. 4. The proposed CRP modelling.

3.1. Problem modelling and formulation

The CRP is modelled as a bi-objective problem that aims to (I) reduce the amount of communicated data and (II) achieve maximum accuracy in FL using heterogeneous devices with varying performances. It is assumed that the *local* ANN model trained by each client and the *global* one in the server have the same structure (i.e., layers, neurons, etc.). The ANN model is a tuple $L = (l_1, \dots, l_{|L|})$ containing vectors of weights, where vector l_i $(i = 1, \dots, |L|)$ represents the weights of the connections between the neurons of two consecutive layers of the ANN. We also distinguish the connection between the bias and the next layer of the ANN as a different vector of weights (see Fig. 3).

The training dataset Δ contains $|\Delta|$ data samples (e.g., images) that are equally and randomly split over the N clients. Once the algorithm achieves T aggregations, a test dataset Ξ is used on the server to measure the accuracy of the global model. This work does not address the issue of an unbalanced and *non-i.i.d.* [7,26] dataset to avoid adding noise to the analysis of devices' heterogeneity.

In FL, different techniques exist for dealing with the devices' heterogeneity. This work considers a variation of the FedAvg algorithm. In vanilla FedAvg (see Algorithm 1), each global aggregation t, m devices are randomly selected to participate in the communication round. The server waits for everyone to finish before starting the next round of communication, i.e., they are synchronous. In our proposal, these clients are assigned a maximum time window proportional to the number of training steps performed by the reference device with speed 1.0. Therefore, each client k will perform a different number of local steps $(E \cdot \upsilon_i)$ proportional to its normalised computation speed υ_i. In this work, each device's speed takes as a reference an ideal device of a speed of 1.0, which corresponds to a common type of client (i.e., standard). The CRP optimises the following parameters (see Fig. 4):

1. First, CRP optimises the number of selected clients m participating in each communication round, reducing communications proportionally when fewer devices are chosen. Clients are picked from all available ones in each communication round, as the vanilla FedAvg algorithm does [8].

2. CRP also optimises the integer parameter E, which indicates the number of local steps performed by the selected clients S_t between two global aggregations. The more local steps are performed, the fewer communication rounds are needed to train using the same data samples.

3. Then, for each vector of weights l_i, an integer y_i is used to encode the percentage of weights that will be sent from the local devices to the server during the communication round (sparsification). The fewer weights are transmitted, the less communication occurs proportionally, although it may impact accuracy.

4. Similarly, for each l_i, an integer q_i represents the number of bits used to encode the weights when they are sent to the server during the communication round (quantisation). Using fewer bits to encode weights reduces communication proportionally but may affect accuracy.

5. The mini-batch size, i.e., the number of samples to work through before updating the model weights, influences the speed and convergence of the ANN model. Therefore, the CRP model includes a problem variable b to be optimised, where $b \in \{b_1, \dots, b_B\}$ is the size of mini-batch used to split the dataset Δ, and B is the number of values that b can take. Typically, larger batch sizes progress faster, and their gradient estimations are closer to the true gradient [35]. However, when the batch size exceeds a threshold, it leads to a loss in generalisation performance (accuracy in the test dataset) because the generated noise in stochastic gradient descent (SGD) is insufficient to exit from local optima [35]. In contrast, smaller batch sizes are noisy and train slower but usually converge to good solutions. However, too small batch sizes provoke too much noise, preventing gradient descent from fully converging to an optimum [35].

6. Setting the right mini-batch size is a complex problem affecting the model's statistical accuracy (generalisation) and hardware efficiency [28]. Thus, to help solve the convergence dilemma, the learning rate $\eta \in \{\eta_1, \dots, \eta_A\}$ is also considered an optimisation variable, where A is the number of possible values that η can take.

7. After the local training, each client k sends its weights L_t^k to the server. The CRP modelling considers a variable $\phi \in \{\phi_1, \dots, \phi_Z\}$ that encodes the technique used for the aggregation of the received weights/gradients which were trained using a different number of local steps. Z is the number of possible strategies that can be applied. Assigning greater importance to the weights of models that have undergone more steps could be advantageous, and we want to analyse whether this holds in practice.

8. Also, a variable $\sigma \in \{\sigma_1, \dots, \sigma_u\}$ is considered to decide which technique, among u ones, will be applied for aggregating the 0s generated during the sparsification step. Remember that the sparsification technique only sends the weights above a threshold, so we assign zero value to the rest. It is expected that the optimal strategy is skipping the zeros, but we want to analyse if this holds in practice and what impact it has on the results compared to other techniques.

9. Finally, a binary parameter g is considered to decide whether the selected clients send the weight tuple L_t^k ($g = False$) or the accumulated gradients $L_t^k - L_{t-1}^k$ ($g = True$). One might think that sending weights and accumulated gradients are the same but give different results when using sparsification. For example, the sparsification technique selects the higher absolute values, and doing that from the neural network weights differs from picking the higher absolutes values from the accumulated gradients (the variation since the last aggregation). In the first case, the new weights replace the previous global weights, which may get zero values if the sparsification technique has been used. In the second case, the accumulated gradients are summed to the global weights, which do not get zero values if they did not have them before. We want to analyse which of these two techniques performs better in practice.

Two of the mentioned parameters are of vector type ($q_1 \dots q_{|L|}$ and $y_1 \dots y_{|L|}$), optimising a different value for each weight vector. The other seven are of integer type. Five parameters aim to reduce communication overload and enhance accuracy (m, E, b, $q_1 \dots q_{|L|}$ and $y_1 \dots y_{|L|}$). The four remaining parameters (η, g, ϕ, and σ) are dedicated only to improving accuracy.

The number of global aggregations T (see Eq. (5)) performed in a global epoch (i.e. using all data from all selected and ideal devices) is determined by the size of the training dataset $|\Delta|$, the size of the mini-batch chosen b, the total number of clients N and the number of local steps E performed by the reference device with speed $\upsilon = 1.0$.

The CRP mathematical formulation is presented in Eqs. (1)–(10), where the goal is to find the optimal hyperparameters that (I) minimise the communication load (f_1) and (II) maximise the model's accuracy (f_2). The parameters R and H represent the normalised total amount of data received and sent by the clients, respectively.

To optimise:

$$\min_{x=\{x_1,\dots,x_{(|L|\cdot 3)+3}\}} f_1(x)$$

$$f_1(b, m, E, y_1, \dots, y_{|L|}, q_1, \dots, q_{|L|}, |l_1|, \dots, |l_{|L|}|) = \frac{R+H}{2} \tag{1}$$

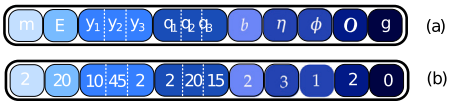


Fig. 5. CRP solution: (a) standard and (b) example.

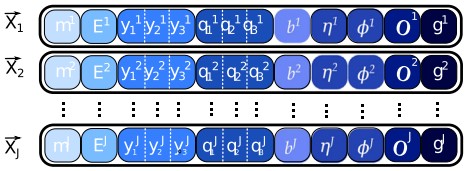


Fig. 6. Standard FL-OPT population.

$$\min_{x=\{x_1,\ldots,x_{|w_T^*|}\}} \quad -f_2(x)$$

$$f_2(w_T^*) = ACC(w_T^*, \Xi) \tag{2}$$

Where:

$$R = \frac{T}{T_{max}} \cdot \frac{m}{N} \qquad 0 < R <= 1 \tag{3}$$

$$H = \frac{T}{T_{max}} \cdot \frac{m}{N} \cdot \sum_{i=1}^{|L|} \frac{q_i}{32} \cdot \frac{100 - y_i}{100} \cdot \frac{|l_i|}{\sum_{z=1}^{|L|} |l_z|} \qquad 0 < H <= 1 \tag{4}$$

$$T = \left\lceil \frac{|\Delta|}{b \cdot N \cdot E} \right\rceil \tag{5}$$

$$T_{max} = \left\lceil \frac{|\Delta|}{b_1 \cdot N \cdot 1} \right\rceil \tag{6}$$

Subject to:

$$0 < m \leq N \qquad \forall\, m, N \in \mathbb{N} \tag{7}$$

$$1 \leq E \leq \frac{|\Delta|}{b \cdot N} \qquad \forall\, E \in \mathbb{N} \tag{8}$$

$$1 \leq q_i \leq 32,\ i = 1, \ldots, |L| \qquad \forall\, q_i \in \mathbb{N} \tag{9}$$

$$0 \leq y_i \leq 100,\ i = 1, \ldots, |L| \qquad \forall\, y_i \in \mathbb{N} \tag{10}$$

Algorithm 2 The Federated Learning Optimiser.

```
Set F = {F_1, ..., F_K} the non-dominated fronts;
P ← Random_Initialisation(J);
P, ACC_P, COM_P ← Evaluation(P);
while termination criterion not reached do
    A ← Selection_Based_On_Dominance(P, ACC_P, COM_P);
    B ← Crossover(A);
    Q ← Mutation(B);
    G ← EDA_Intensification(P, μ, λ);
    M, ACC_M, COM_M ← Evaluation(Q ∪ G);
    P ← P ∪ M;
    ACC_P ← ACC_P ∪ ACC_M; COM_P ← COM_P ∪ COM_M;
    F ← Non_Dominated_Sorting(P, ACC_P, COM_P);
    P' ← ∅;
    i ← 1;
    while (|P' ∪ F_i| ⩽ J and i ⩽ K) do
        P' ← P' ∪ F_i;
        i ← i + 1;
    end while
    F_i ← Sort_Crowding_Comparison(F_i, ACC_P, COM_P);
    P ← P' ∪ F_i[1 : (J − |P'|)];
end while
```

Fig. 5(a) represents an example of the encoding of a general solution for the CRP when dealing with an ANN model with three weight vectors ($|L| = 3$), while Fig. 5(b) illustrates a possible solution that uses two clients and performs 20 locals steps. In this example, the sparsification is done by sending 90%, 55% and 98% of the weights of the 1st, 2nd and 3rd weight vector, respectively. As to the quantisation, it is performed by encoding the weights of the 1st, 2nd and 3rd weight vector using 2, 20 and 15 bits, respectively. A mini-batch of size corresponding to the 3rd setting and a learning rate of the 4th configuration are applied. It applies the 2nd aggregation method and the 3rd approach for dealing with the 0s resulting during the sparsification. Finally, each client sends its weights (g = *False (0)*) instead of the accumulated gradients (g = *True (1)*).

3.2. The federated learning optimiser

We considered various alternatives in our pursuit of objectives, including exhaustive hyperparameter optimisation, automated tools like IRace [36] for efficient hyperparameter search, and pre-training compression for reducing model size and communication needs. However, each of these approaches has its limitations. Exhaustive exploration becomes resource-prohibitive in high-dimensional spaces. Automated tools may not efficiently capture the intricate relationships between hyperparameters. Model compression may compromise accuracy. We chose NSGA-II because it can efficiently find high-quality solutions and provide a Pareto front, representing the optimal trade-off. NSGA-II's non-uniform exploration quickly converges towards promising regions, making it suitable for the complex FL space. However, future work may explore these alternatives individually or in combination to further enhance our understanding and address the challenges of FL.

The CRP solver proposed in this work, called Federated Learning Optimiser (FL-OPT), consists of an NSGA-II [22] coupled with an EDA [23,25] (see Algorithm 2). The devised approach optimises the two objectives defined by Eqs. (1) and (2). This is done by evolving a set of solutions $x = (m, E, y_1, \ldots, y_{|L|}, q_1, \ldots, q_{|L|}, g, \phi, \mathcal{O}, \eta, b)$ (see Section 3.1). First, the FL-OPT randomly initialises a population P of J individuals. Then, it enters a loop until a stop criterion is reached. Next, it iteratively applies (I) a binary tournament selection, (II) crossover, (III) mutation to generate a population Q of J offspring, (IV) an intensification is performed on the μ best individuals using an EDA (UMDA) to generate a population G of λ offsprings. Finally, the union of parents and offspring, i.e. $P = P \cup Q \cup G$ will undergo (V) a replacement to decide the components of the new population P' of size J that will go through the next iteration. The following provides more insight into each of the phases performed by the devised FL-OPT.

3.2.1. Initialisation and selection

The initialisation creates a population P of J solutions. Fig. 6 represents a typical representation of the population of solutions evolved by the proposed FL-OPT in the case of a model of $|L| = 3$.

The selection applies a binary tournament to select a set A of $|P|/2$ pairs of parents from the population P to apply crossover and mutation. The selection criterion is based on the crowding comparison devised in the original NSGA-II (see [22] for more details).

3.2.2. Crossover

During this phase, each selected pair of parents from the set A, say x_1 and x_2, has a probability P_c to undergo a uniform crossover to generate two offspring x_1' and x_2' and the parents are kept untouched with probability $1 - P_c$. In the uniform crossover, for each gene, an offspring receives the gene from the first parent while the other offspring receives the gene from the other parent. Fig. 7 illustrates a uniform crossover applied on two parents in the case of $|L| = 3$. Once all the pairs are processed, a new set B of J crossed offspring is created.

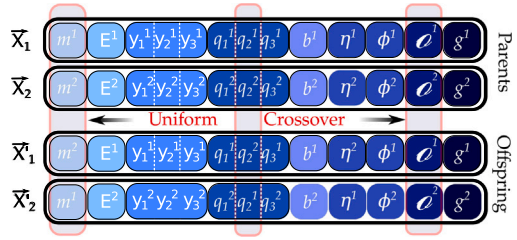


Fig. 7. Uniform crossover implemented in the FL-OPT.

Fig. 8. Uniform mutation implemented in the FL-OPT.

Fig. 9. EDA (UMDA) intensification in the FL-OPT.

3.2.3. Mutation

During this step, each gene of every offspring in the set B may undergo a uniform mutation (see Fig. 8) according to the probability P_m (otherwise, they are kept untouched). Technically, the mutation consists in randomly affecting the gene, assigning a new value from the range of valid values permitted for that gene. This step produces a new set Q of J mutated offspring. Fig. 8 sketches how uniform mutation is applied on a solution x' to produce a new offspring x'' acting on a model with $|L| = 3$.

3.2.4. Intensification

Although the CRP is bi-objective, it is clear that solutions providing a substantial communication reduction without enhancing or at least maintaining a decent accuracy are of no interest. To overcome this constraint, the FL-OPT executes an EDA (UMDA) (see Fig. 9) over the population P to optimise individuals with better accuracy. During this step, a normal random number generator is initialised with a certain mean $\overline{x} \dots \overline{x_{|d|}}$ and a standard deviation $\sigma_1 \dots \sigma_{|d|}$ for each gene. It sorts the population from highest to lowest accuracy, selects the μ first individuals, and samples λ new offspring G from the estimated distribution of their gene values. Then, the average of the μ best individuals is computed to estimate the variance of each gene of each individual. Finally, the distribution parameters are updated.

3.2.5. Replacement

The replacement is performed using the non-dominated sorting heuristic and the crowding-comparison operator presented in the original NSGA-II to decide the composition of the population P' that will undergo the next iteration. The non-dominated sorting results in a set $F = \{F_1, \dots, F_K\}$ of K non-dominated fronts of increasing rank i, where $i \in \{1, \dots, K\}$. Having F_1 the front that is not dominated by any other one, while the remaining fronts are dominated by all those with a lower rank. Based on the crowding-comparison operator, the FL-OPT favours solutions of lower rank if the solutions being compared belong to different fronts. However, if the solutions come from the same front, it favours the solution having a higher crowding distance. For more details about the non-dominated-sorting and crowding-comparison operators, one should refer to the NSGA-II original work [22].

4. Experimentation and discussion

In this section, first, the settings used for the experimental study are explained. Next, the three performed studies are introduced. Finally, the obtained results in each experiment are analysed and discussed.

4.1. The hardware and software settings

The proposed algorithm and problem are implemented in `Python` version 3.8.8 and the experiments are run in a computation cluster using two different kinds of machines: `(I)` 24 × Bull R282-Z90 nodes with 128 cores (AMD EPYC 7H12 @ 2.6 GHz), 2 TB of RAM, InfiniBand HDR200 network, 3.5 TB of local-scratch disks and, `(II)` 4 × DGX-A100 nodes with 8 GPUs (A100 Tensor Core), 1 TB of RAM, InfiniBand FDR40 network and a 14 TB of local-scratch. Each experiment setup with a different seed uses 120 cores and 400 GB of RAM.

In the experimentation, two types of ANN topologies are used, one fully connected with 33,400 weights ($|L| = 4$, with 32928, 42, 420 and 10 weights in each weight vector) and one convolutional with 887,530 weights ($|L| = 12$, with 800, 32, 25600, 32, 18432, 64, 36864, 64, 802816, 256, 2560 and 10 weights in each weight vector). The same seed is always used to initialise the initial weights of the ANN. Two different datasets are used, the first is the `MNIST` dataset containing 70,000 images of handwritten digits, and the second is the `Fashion-MNIST` dataset containing 70,000 images of Zalando's article images. Both datasets are commonly employed in the literature. They are well-suited for our intended purpose as they balance the time required for training the model with each configuration and achieve representative results for comparison. The second dataset presents a slightly more intricate challenge than the first, which enables us to scrutinise two problems with varying levels of complexity. Additionally, utilising fully connected and convolutional neural network topologies, the second one incorporating more layers and parameters, allows us to conduct a more comprehensive analysis. Consequently, this approach permits us to analyse the convergence of solutions across different problems and complexities, gaining insights into their similarities and disparities. Experiments are referred to with the following abbreviations: Fully Connected/MNIST (FC-MNIST), Fully Connected/FASHION (FC-FASHION), Convolutional/MNIST (C-MNIST), and Convolutional/FASHION (C-FASHION). The size d of the solutions in the fully connected model is 15, and in the convolutional model, is 31. Each experiment is executed 30 times using 30 different seeds and a termination criterion of 50 iterations, using the two algorithms NSGA-II and FL-OPT, i.e. without and with intensification. A population of 100 individuals is used, $P_c = 0.9$ and $P_m = (1/d)$. The EDA algorithm uses $\mu = 5$, $\lambda = 10$. The results of NSGA-II and FL-OPT algorithms are compared in each experiment. Therefore, to ensure fairness in the total number of evaluations performed by each solver considered during the comparison, the results of NSGA-II at iteration 50 are experimentally compared with those of the FL-OPT at iteration 45 (i.e. 5,000 and 4,950 evaluations, respectively) to take into account the evaluations performed by the EDA.

A higher mutation rate promotes exploration by introducing variability into existing solutions, enabling the search for new areas in the solution space. Conversely, a higher recombination probability facilitates exploitation by favouring the combination of beneficial characteristics from existing solutions, generating offspring with advantageous inherited traits. Also, larger populations favour exploration by diversifying the search space, while smaller populations emphasise exploitation by intensifying focus on promising solutions. The chosen parameters for the NSGA-II operators are the standard in the literature. The proposed intensification operator aims to enhance the exploitation

Table 1
Experimentation parameters.

Description	Parameter	Fully connected ANN	Convolutional ANN
Fixed parameters			
–	Optimiser	Adam	SGD
–	Loss	CategoricalCrossentropy	CategoricalCrossentropy
Total number of clients	N	4	4
Total vectors of weights	$\|L\|$	4	12
Weights in each vector	$\|l_1\| \dots \|l_{\|L\|}\|$	32928, 42, 420, 10	800, 32, 25600, 32, 18432, 64, 36864, 64, 802816, 256, 2560, 10
Trainable weights	$\sum_{z=1}^{\|L\|} \|l_z\|$	33,400	887,530
Normalised computation speed	υ in heterogeneous case	{2.0, 1.0, 0.75, 0.5}	{2.0, 1.0, 0.75, 0.5}
Dataset	Δ	∈ {Mnist, Fashion-Mnist}	∈ {Mnist, Fashion-Mnist}
Parameters to optimise			
Number of selected clients	m	∈ [1..4]	∈ [1..4]
Learning rate	η	∈ {0.0005, 0.001, 0.0015, 0.002}	∈ {0.005, 0.01, 0.015, 0.02}
Mini-batch size	b	∈ {4, 8, 16, 32}	∈ {4, 8, 16, 32}
Number of local steps	E	∈ [1..1000]	∈ [1..1000]
Number of bits (quantisation)	$\{q_1 \dots q_{\|L\|}\}$	∈ [1..32]	∈ [1..32]
Percentage of weights sent (sparsification)	$\{y_1 \dots y_{\|L\|}\}$	∈ [0..50]	∈ [0..50]
Sending weights (False) or gradients (True)	g	∈ {False, True}	∈ {False, True}
Heterogeneous aggregation technique	ϕ	∈ {Equal, Proportional, InvProportional}	∈ {Equal, Proportional, InvProportional}
Zeros aggregation technique	$\varnothing$	∈ {Ignore, Skip, Force}	∈ {Ignore, Skip, Force}

of solutions in the Pareto front with the highest accuracy, accelerating their convergence towards optimal solutions. The selection of λ and μ values was based on preliminary tests.

We propose three strategies for dealing with zeros to analyse their performance. The first performs standard averaging while ignoring sparsification (*Ignore*). The second one is averaging by skipping the zeros (*Skip*). The third is to force a position to zero in all received weights/gradients if its value is zero for any client (*Force*). We also study the performance of three different techniques for weighted aggregation. The first one is the *equal* approach, which aggregates the results of heterogeneous devices equally, regardless of whether some devices have performed more local steps than others. The second technique is *proportional* and assigns a weight to each device proportional to the number of local steps it has performed. The third approach, *invProportional*, assigns a weight to the devices inversely proportional to the number of local steps performed. Four possible values are chosen for mini-batches and learning rates depending on the topology of the model and the optimiser used. We chose [0.0005, 0.001, 0.0015. 0.002] as potential learning rates for the fully connected network with Adam optimiser and [0.0005, 0.001, 0.0015. 0.002] for the convolutional network with SGD optimiser. We chose [4, 8, 16, 32] as candidate mini-batches. These values are selected based on our own experience and observation of the work of other researchers. Table 1 shows all the parameters used.

Each experiment uses four clients ($N = 4$) with different processing speeds (2.0, 1.0, 0.75 and 0.5). The training dataset ($|\Delta| = 60{,}000$ images) is distributed between the clients randomly and equally. During the learning process, each client can only access its private dataset. Each ANN trains for the equivalent of training for an epoch on the ideal device with a speed of 1.0. For instance, if there are 60,000 images that use a minibatch of 8, then the results are divided between the four clients, resulting in 1,875 mini-batches stored in each client. If 500 local steps are used, then four global aggregations are performed (of 500, 500, 500, and 375 local steps, respectively). Each local step corresponds to training with one minibatch on each selected client. If a device has a speed of two, it will perform 1000, 1000, 1000, and 750 local steps in each global aggregation. Not all clients are selected in all global aggregations, but the number of global aggregations is the same regardless of which clients are selected. The final models obtained are evaluated with the 10,000 images of the test dataset Ξ. One should also note that all our results have been confirmed using a Wilcoxon test with Bonferroni correction and a significance level of 0.025.

Table 2
Comparison of the best accuracy solutions found in the Pseudo-optimal Pareto front in the case of FL-OPT and NSGA-II.

Algorithm	ACC	COMM	ACC	COMM
FC-MNIST			C-MNIST	
FL-OPT	**0.9597**	**0.0006**	**0.9904**	**0.0012**
NGSA-II	0.9589	**0.0006**	0.9898	0.0014
FC-FASHION			C-FASHION	
FL-OPT	**0.8522**	0.0573	**0.8846**	0.5594
***FL-OPT**	0.8499	**0.0001**	0.8775	**0.0008**
NGSA-II	0.8492	**0.0001**	0.8772	0.0009

4.2. Studies

Three different studies are analysed. In the first study, we compared two multi-objective approaches: the vanilla NSGA-II and our proposed approach incorporating an EDA as a new operator. This analysis aims to validate that the new operator significantly enhances the algorithm's performance. The pseudo-optimal Pareto fronts obtained by the algorithms and the evolution of accuracy, level of communications, and relative hypervolume are compared.

In the second study, the accuracy and communication provided by our approach are compared against two predefined solutions using a configuration with a 100% communication level (comm = 1) and another with a 50% communication level (comm = 0.5). The goal is to analyse whether the proposal is not only able to obtain solutions with less communication but also to maintain and even improve the level of accuracy. After analysing the performance of our proposal compared to other approaches, in our third study, we aim to examine the configurations obtained (parameters) in the problem domain. To achieve this, we analyse the distribution and correlation of the resulting parameters of the solutions in the final Pareto front.

4.3. Evaluation of multi-objective algorithms

This study compares the pseudo-optimal Pareto fronts obtained by the algorithms and the evolution of accuracy, level of communications, and relative hypervolume.

In multi-objective optimisation, the Pareto front [22] is a set of non-dominated solutions chosen as optimal if no objective can be improved without sacrificing at least one other objective. We call the set of non-dominated solutions that each algorithm found when run 30 times

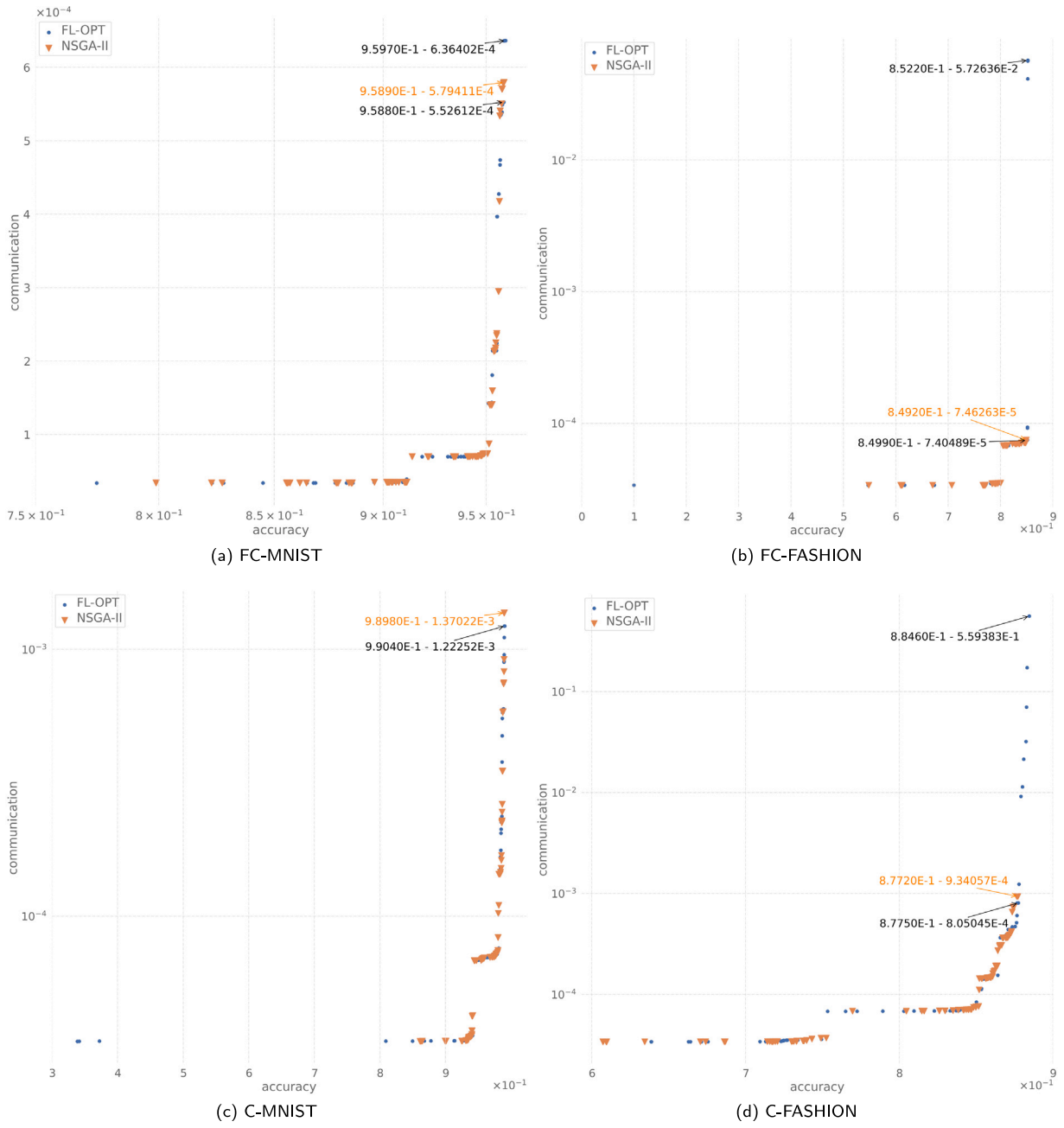


Fig. 10. Pseudo-optimal Pareto front of the Fully Connected ANN (a) (b) and the Convolutional ANN (c) (d) using intensification EDA (FL-OPT) and not using intensification (NSGA-II).

with 30 different seeds a pseudo-optimal Pareto. The hypervolume metric [37,38] represents the size of the covered or dominated objective space compared to the one of the reference Pareto front. It is a convergence and diversity performance metric for indicating the quality of a non-dominated Pareto set. It considers the points' proximity to the reference Pareto front, diversity, and spread. In this case, the reference Pareto used to calculate the relative hypervolume is the pseudo-optimal Pareto when computing the non-dominated solutions of all the runs (using the 30 seeds) of both algorithms (same dataset and neural network topology). Ideally, the relative hypervolume is calculated using the Pareto optimal front, which, in this case, is unknown.

Fig. 10 shows the pseudo-optimal Pareto front. The results of the FL-OPT and NSGA-II are compared using the fully connected ANN *(a)* and *(b)* and the convolutional ANN *(c)* and *(d)* for the two data sets under study, MNIST *(a)* and *(c)* and Fashion-MNIST *(b)* and *(d)*.

These figures show that in all cases: FC-MNIST, FC-FASHION, C-MNIST, and C-FASHION, the FL-OPT algorithm got a non-dominated solution with better accuracy than the NSGA-II algorithm (see Table 2). In the case of FC-MNIST, the best accuracy found by the FL-OPT algorithm [0.9597, 0.0006] improves the accuracy compared to the best one found by the NSGA-II [0.9589, 0.0006] with almost the same cost in communications. In the case of FC-FASHION, the first got [0.8522, 0.0573] and the latter [0.8492, 0.0001], improving the accuracy but not the communication level. However, in this case, we can find another solution in the FL-OPT pseudo-optimal Pareto that improves on both aspects [0.8499, 0.0001]. In the case of C-MNIST, the first got [0.9904, 0.0012] and the latter got [0.9898, 0.0014], improving accuracy and reducing communications. In the case of C-FASHION, the first got [0.8846, 0.5594], and the latter [0.8772, 0.0009], improving the accuracy but not the communication level. However, as in the FC-FASHION case, we can find another solution

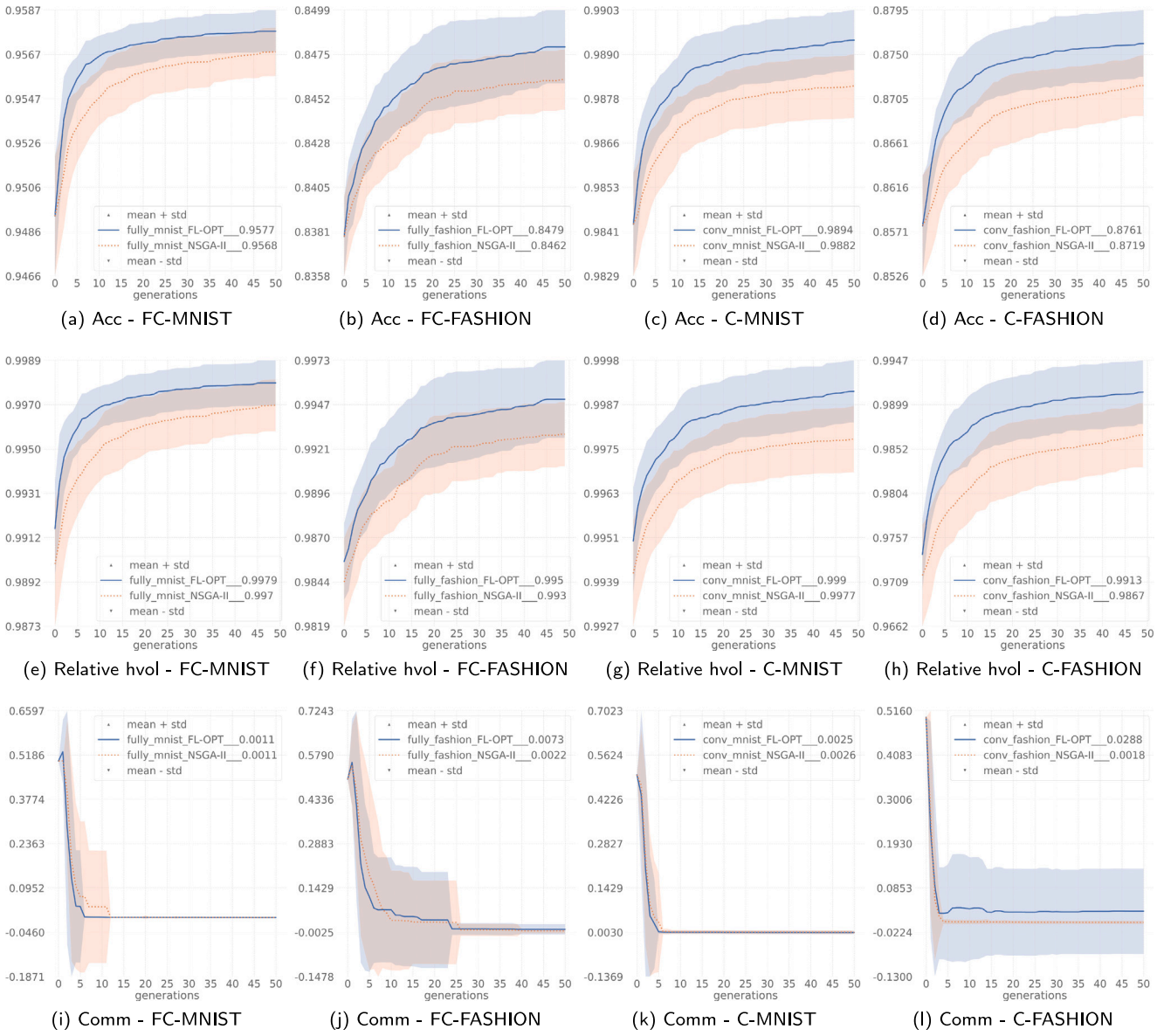


Fig. 11. Evolution of accuracy, relative hypervolume and communication of the optimal Pareto for the four experiments and two algorithms. Each column represents a configuration with one dataset and one NN topology when using intensification EDA (FL-OPT) and not using intensification (NSGA-II).

in the FL-OPT pseudo-optimal Pareto that enhances both [0.8775, 0.0008]. The results also show that the FL-OPT algorithm explores the Pareto front more efficiently, increasing the diversity of solutions and finding more non-dominated solutions on the extremes of the Pareto front.

Fig. 11 displays the mean plus/minus the standard deviation of the evolution of accuracy, relative hypervolume, and communication level on the population of non-dominated solutions of each algorithm NSGA-II (None) and FL-OPT (EDA) over the 50 iterations in the four experiments (FC-MNIST, FC-FASHION, C-MNIST, and C-FASHION). Note that the mean minus the standard deviation can result in negative numbers, but no solution can have a negative accuracy value [0,1] or communication [0,1].

FL-OPT consistently outperforms vanilla NSGA-II in all experiments, achieving higher accuracy and hypervolume values. The mean of communication level of the optimal Pareto remains almost the same for both algorithms when using the MNIST dataset and increases very little in the case of FL-OPT when using the Fashion-MNIST dataset compared to the NSGA-II. In the case of C-FASHION, the standard deviation of the communication of the solutions found in the FL-OPT Pareto Front is wider because more non-dominated solutions have been found with excellent accuracy but with high communication, which makes this deviation increase (see Fig. 10(d)). However, the FL-OPT algorithm also found solutions with better accuracy and lower communication than the best ones using the NGSA-II.

The slight increase in the average communication level of the non-dominated solutions of the FL-OPT Pareto Front compared to the one in NSGA-II is due to FL-OPT finding more diverse non-dominated solutions, some of them with better accuracy and higher communication, which makes the average communication level of the Pareto Front increase. However, this does not indicate that the solutions found by FL-OPT have a higher communication level than those of NSGA-II since, in all cases, FL-OPT finds solutions with better accuracy and the same or even less communication level than the higher accuracy solutions found by the NSGA-II. Furthermore, FL-OPT obtains a higher relative hypervolume in all cases than NSGA-II. The latter indicates that the solutions cover or dominate a more significant part of the reference pseudo-optimal Pareto front (i.e., the non-dominated solutions of all the runs of both algorithms), favouring the diversity and spread of the non-dominated solutions.

Table 3
Accuracy and communications level of top 15 accuracy solutions of FL-OPT Pareto front and solutions using a 100% and 50% communication setting.

Instance	Accuracy				Communications			
	Mean ± Std	Median	Max	Min	Mean ± Std	Median	Max	Min
FC-MNIST								
100%	0.9505 ± 0.0028	–	0.9547	0.9430	1.0000 ± 0.0000	–	1.0000	1.0000
50%	0.9458 ± 0.0022	–	0.9485	0.9410	0.5000 ± 0.0000	–	0.5000	0.5000
Top 15 ACC solutions in Pareto	**0.9577 ± 0.0009**	**0.9577**	**0.9597**	**0.9560**	**0.0011 ± 0.0008**	**0.0005**	**0.0038**	**0.0005**
FC-FASHION								
100%	0.8421 ± 0.0058	–	**0.8544**	0.8323	1.0000 ± 0.0000	–	1.0000	1.0000
50%	0.8411 ± 0.0038	–	0.8477	0.8297	0.5000 ± 0.0000	–	0.5000	0.5000
Top 15 ACC solutions in Pareto	**0.8479 ± 0.0019**	**0.8490**	0.8522	**0.8442**	**0.0073 ± 0.0170**	**0.0001**	**0.0635**	**0.0001**
C-MNIST								
100%	0.9845 ± 0.0020	–	0.9879	0.9782	1.0000 ± 0.0000	–	1.0000	1.0000
50%	0.9802 ± 0.0017	–	0.9832	0.9752	0.5000 ± 0.0000	–	0.5000	0.5000
Top 15 ACC solutions in Pareto	**0.9894 ± 0.0008**	**0.9889**	**0.9904**	**0.9872**	**0.0025 ± 0.0052**	**0.0007**	**0.0300**	**0.0006**
C-FASHION								
100%	0.8615 ± 0.0055	–	0.8675	0.8483	1.0000 ± 0.0000	–	1.0000	1.0000
50%	0.8464 ± 0.0055	–	0.8529	0.8314	0.5000 ± 0.0000	–	**0.5000**	0.5000
Top 15 ACC solutions in Pareto	**0.8761 ± 0.0034**	**0.8779**	**0.8846**	**0.8687**	**0.0288 ± 0.1038**	**0.0012**	0.5594	**0.0005**

In summary, FL-OPT finds non-dominated solutions with higher accuracy than NSGA-II, achieving a pseudo-optimal Pareto front with higher hypervolume. The solutions from FL-OPT cover a larger portion of the reference pseudo-optimal Pareto front and exhibit greater diversity than those obtained with vanilla NSGA-II. These improvements are observed from the initial to the final generation, confirming the hypothesis that optimising a specific part of the Pareto front in a targeted way leads to higher maximum accuracy with minimum compromise of the communication level.

4.4. Analysing the improvement in accuracy and level of communications

This study analyses if the proposed approach (FL-OPT) enhances accuracy compared to configurations with 100% (comm = 1) or 50% communication (comm = 0.5) in addition to reducing communications. We say that we use 100% communication when using a configuration allowed within the algorithm's parameters with a maximum communication of 1 (see Algorithms (3), (4), and (6)), i.e., in each round of communication, we use all available devices ($m = N = 4$), each device performs only one local step ($E = 1$), no sparsification ($y_1, \ldots, y_{|L|} = 0$) or quantisation ($q_1, \ldots, q_{|L|} = 32$) is used, and the smallest allowed batch size is used (b_1).

A set of experiments are carried out using a high level of communications (100% and 50%) and the parameters with the most converged values in the FL-OPT final Pareto front (see Section 4.5), i.e., $g = 0, \phi = 0, \sigma = 0, \eta = 3$ (0.002 in fully connected and 0.02 in convolutional). Two configurations are set with 100% and 50% communication levels, i.e., $b = 0$ (4) and $b = 1$ (8), respectively, and evaluate FC-MNIST, FC-FASHION, C-FASHION, and C-MNIST in each case using 30 different seeds. In this case, each seed is used for initialising ANN weights in each training. The results can be seen in Table 3. The first thing we see in the results is that the configuration with 100% communication achieves better accuracy in all cases than the configuration with 50% communication. However, it is also seen that the FL-OPT Pareto front solutions always manage to reduce communication and improve accuracy compared to the two previous cases. The results of the experiments show that both the mean and the median of the 15 best solutions of the FL-OPT Pareto front improve the average accuracy obtained when using the 100% configuration. But even better, in all cases, the minimum accuracy of the 15 best FL-OPT Pareto front solutions improves the average accuracy of the 100% configuration. Even more so, in C-MNIST, the minimum accuracy FL-OPT is practically the same as the maximum one in the 100% configuration. Moreover, the minimum accuracy in FC-MNIST and C-FASHION is even higher than the maximum in the 100% configuration. Our proposal finds a large set of solutions that reduce the level of communication and improve the accuracy at different levels compared to the maximum communication setting. It allows the researcher to understand better which configuration is the most suitable for a particular problem.

Next, the level of communication reduction achieved by some of the FL-OPT solutions compared to the 100% communication configuration (comm = 1) is analysed. Fig. 10 displays pseudo-optimal Pareto fronts using FL-OPT for both datasets and ANN topologies *(a)* and *(c)*. *In all experiments (FC-MNIST, FC-FASHION, C-MNIST and C-FASHION), the top accuracy* in the FL-OPT Pareto front (see Table 2) not only improves the accuracy but reduces the communication level compared to 100% (comm = 1). In the case of FC-MNIST, the highest accuracy FL-OPT solution [0.9597, 0.0006] reduces the communication level compared to the 100% communication setting by around 1,572 times (taking into account all the decimals shown in the mentioned figures). In the case of FC-FASHION [0.8522, 0.0573], the reduction is by around 17 times. But, another solution in the FC-FASHION Pareto, [0.8499, 0.0001] also improves the accuracy while reducing communications by around 13,504 times. In the case of C-MNIST [0.9904, 0.0012], the reduction is by around 818 times. In the case of C-FASHION [0.8846, 0.5594], the reduction is by around 1.8 times. Nevertheless, another C-FASHION solution [0.8775, 0.0008] also improves the accuracy while reducing communications by around 1,242 times. Moreover, when the median of the top 15 solutions of the pseudo-optimal Pareto for each configuration is taken. In that case, the next accuracy and communication results are obtained: [0.9577, 0.0005] in FC-MNIST, [0.8490, 0,0001] in FC-FASHION, [0.9889, 0.0007] in C-MNIST and [0.8779, 0.0012] in C-FASHION. These results show an accuracy still higher than the maximum communication configuration but with reductions in the communication level of 2000, 10000, 1429 and 833 times, respectively. These results show that our approach does not just get settings that reduce communication levels during training but also gets more accurate solutions. In conclusion, our proposal achieved solutions with better accuracy than the maximum communication setting while reducing communications by around 1,000 times in most cases and up to about 10,000 times in some cases, which indicates that the noise introduced in the learning process makes it easier to escape from local optima and to generalise better.

4.5. Distribution and correlation of parameters

This study analyses the distribution and Spearman correlation [39] of parameters in the final populations of FL-OPT and two ANN topologies fully connected and convolutional (see Figs. 12, 13, 14, 15, and 16). It also shows the correlation between communication and the

(a) FL-OPT - PseudoOpt Pareto - FC-MNIST

(b) FL-OPT - PseudoOpt Pareto - FC-FASHION

(c) FL-OPT - Top 15 Best Acc - FC-MNIST

(d) FL-OPT - Top 15 Best Acc - FC-FASHION

Fig. 12. Parameter distribution in fully connected ANN. All of them refers to FL-OPT results. (a) (c) refer to the MNIST dataset. (b) (d) refer to the Fashion-MNIST dataset. (a-b) refers to the entire final populations obtained from the 30 seeds. (c-d) concerns the 15 solutions with higher accuracy in the final populations of the 30 seeds.

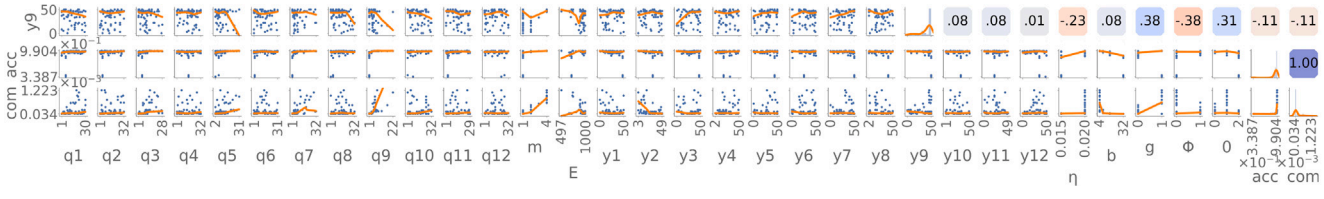


Fig. 13. Parameter distribution of the convolutional ANN using the entire final populations obtained from the 30 seeds when using the FL-OPT algorithm and the Mnist dataset.

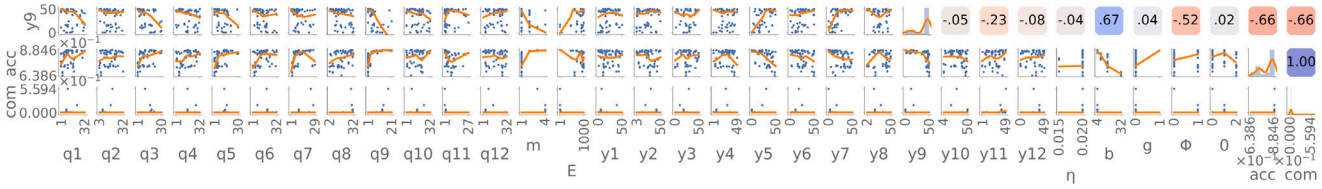


Fig. 14. Parameter distribution of the convolutional ANN using the entire final populations obtained from the 30 seeds when using the FL-OPT algorithm and the Fashion-Mnist dataset.

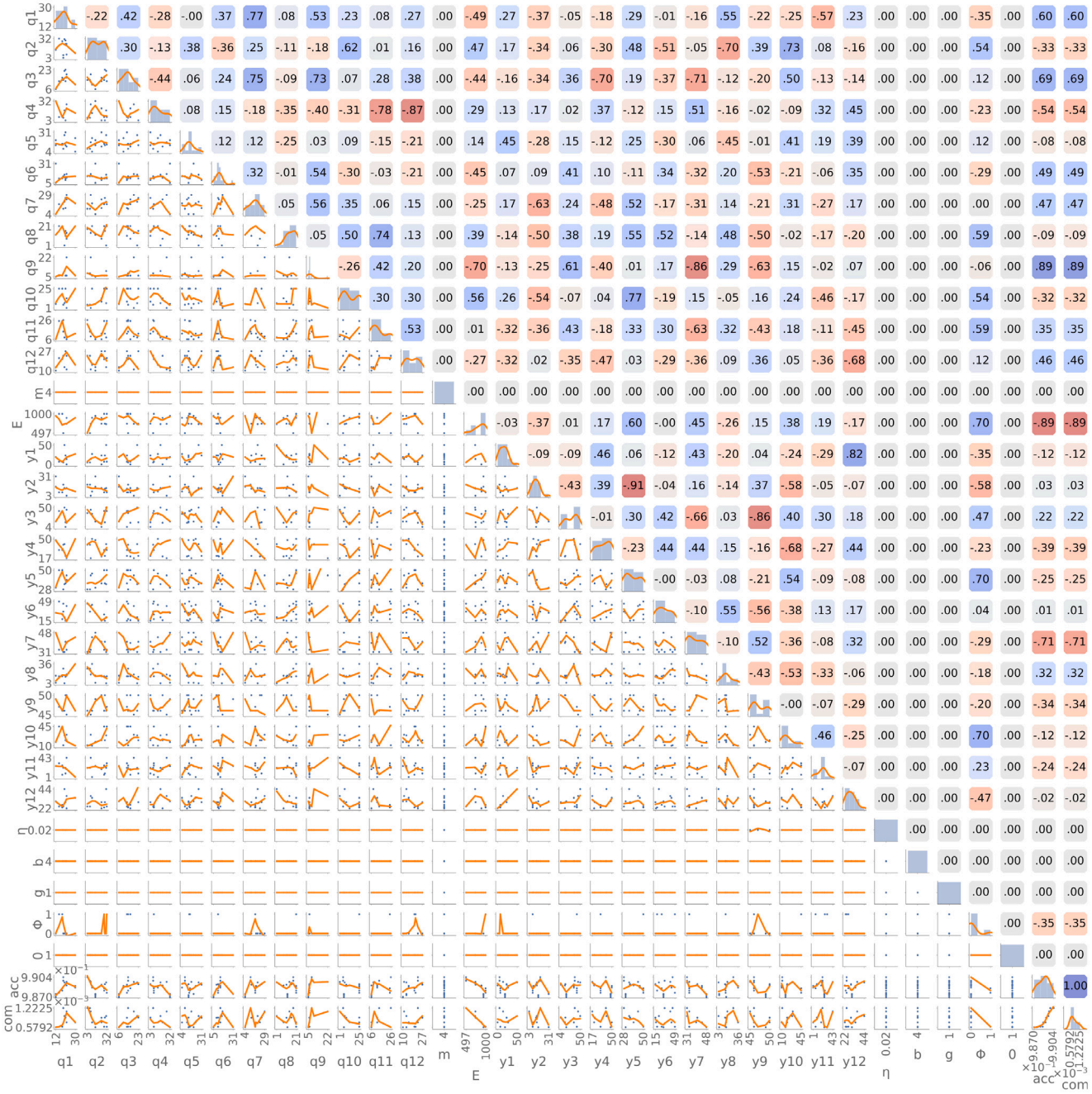


Fig. 15. Parameter distribution of the convolutional ANN using the 15 solutions with the highest precision in the final populations obtained from the 30 seeds when using the FL-OPT algorithm and the Mnist dataset.

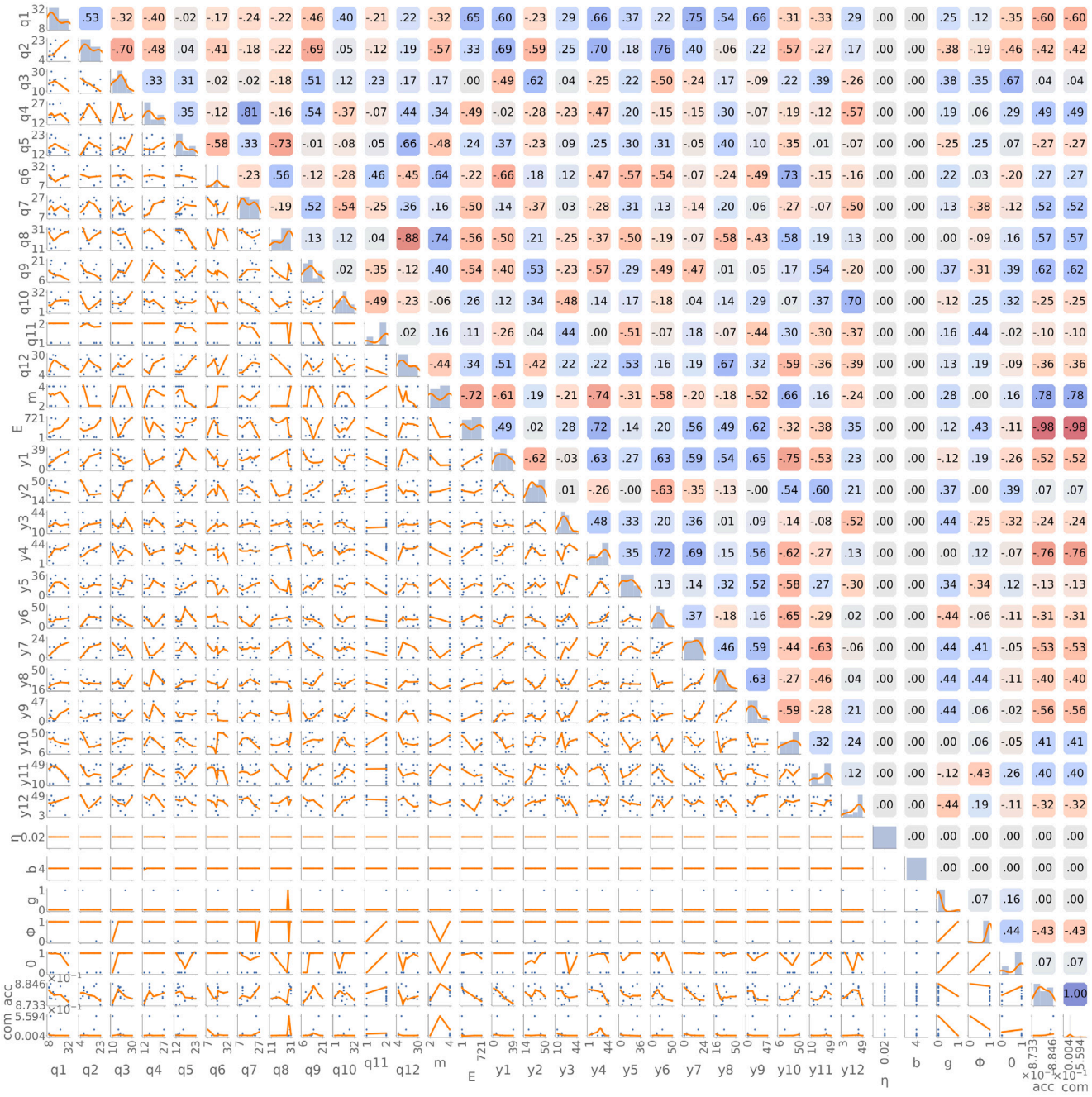


Fig. 16. Parameter distribution of the convolutional ANN using the 15 solutions with the highest precision in the final populations obtained from the 30 seeds when using the FL-OPT algorithm and the Fashion-Mnist dataset.

accuracy of the Pareto solutions. In total, two pairs of subfigures in each figure are shown. The subfigures (a) and (c) refer to the MNIST dataset, and the (b) and (d) refer to the Fashion-MNIST dataset. Subfigures (a) and (b) refer to the entire final population of the FL-OPT pseudo-optimal Pareto front obtained from the 30 seeds. Subfigures (c) and (d) concern the 15 solutions with higher accuracy of the FL-OPT pseudo-optimal Pareto front obtained from the 30 seeds.

In each subfigure, the relationship of each parameter to all the others is noticed. For each parameter, the values in the population that have converged to a specific range of values (the minimum and maximum values) are observed. A scatter plot and a polynomial regression of how each parameter varies compared to the others are seen in the lower-left triangle. In the upper-right triangle, a number indicates the value of the Spearman correlation of each parameter with the others [−1, 1]. Positive values indicate direct correlations and are shown in blue. Negative values show inverse correlations and are shown in red. Lower correlation values have gradient colours up to zero correlation, shown in grey.

First, it is analysed the correlation between parameters and communication level. The weight vectors with the highest number of weights in the fully connected neural network are 1 and 3, and in the convolutional network, they are 3, 5, 7, and 9. Recall that the number of weights in each weight vector in the fully connected topology is [32928, 42, 420, 10], and in the convolutional topology, it is [800, 32, 25600, 32, 18432, 64, 36864, 64, 802816, 256, 2560, 10] (see Table 1). Therefore, it is expected that the parameters that most influence the reduction of communication will be E, m, b, q_1 and q_3 in the case of FC-MNIST and FC-FASHION and E, m, b, q_3, q_5, q_7 in the case of C-MNIST and C-FASHION.

Table 4
Median parameter values for quantisation and sparsification in the top 15 accuracies of FL-OPT Pareto front in FC-MNIST and FC-FASHION.

Number of weights	l_1	l_2	l_3	l_4
	32 928	42	420	10
Quantisation 1...32	q_1	q_2	q_3	q_4
FC-MNIST	**8**	13	12	17
FC-FASHION	**5**	9	17	18
Sparsification 0...50	y_1	y_2	y_3	y_4
FC-MNIST	40	27	14	**46**
FC-FASHION	**43**	26	30	33

In FC-MNIST and FC-FASHION, if looking at the pseudo-optimal Pareto (see Figs. 12(a) and 12(b)) of the whole population, it is seen that b, m, q_1, q_3, y_1 and b are the parameters where there exist the highest correlation with communication getting more solutions with extreme values. These are the parameters where the algorithm focused on reducing communications. The rest of q_i and y_i had more intermediate values to compensate for the earlier effort, achieving high accuracy and reducing communication.

In C-MNIST and C-FASHION (see Figs. 13 and 14), these parameters are q_3, q_5, q_7, q_9, m and b. In this case, the correlation of most layers with the sparsification parameters is more uniform. The algorithm primarily utilised extreme parameter values to maximise communication reduction, compensating with other parameters to maintain higher accuracy. An exception is observed in y_9, where, despite a relatively low/medium correlation value (−11 and -66), most solutions have a high level of sparsification in this layer in both experiments. That is interesting because y_9 influences the layer with more weights, significantly differing from the others. That indicates that a high level of sparsification was performed in this layer without impairing the accuracy, while other parameters compensated for the level of communication. It is also interesting to look at the E parameter, which theoretically also highly influences communication. However, the algorithm had focused on keeping it at high but more evenly distributed values so that its correlation is lower than the previous parameters in all of the experiments and even has zero correlation in C-MNIST and FC-FASHION and direct correlation in FC-MNIST when the correlation was expected to be inverse as it is in C-FASHION (the more local steps, the less the communication).

Next, we examine the convergence tendencies of parameters in experimentation, focusing on the top 15 accuracy solutions within the Pareto front (see Figs. 12(c), 12(d), 15, and 16). A direct correlation is observed between the number of selected devices m and accuracy. However, the highest accuracies only converge to $m = 4$ in the case of C-MNIST. In C-FASHION, m is uniformly between 2 and 4. In FC-FASHION, m has lower values, while in FC-MNIST, m ranges between 2 and 3, with the value three most represented. Noteworthy, using all devices in every communication round is unnecessary to achieve high final accuracy levels. Also, the highest accuracies usually converge to more local steps E (generally around 900) except in C-FASHION, where E values are uniformly between 1 and 721. The algorithm often uses many local steps without harming accuracy. Still, it occasionally reduces this parameter when needed to compensate for other parameters and achieve similar accuracy results, especially when dealing with many layers and a more complex dataset, like C-FASHION.

Besides E and m, some of the parameters that most influence the reduction of communication are q_1 and y_1 in FC-MNIST and FC-FASHION. That is reasonable because they perform quantisation and sparsification on the first layer, which has the highest number of trainable weights. In the 15 highest accuracies of each experiment using FL-OPT, the median values for quantisation (see Table 4) are [8, 13, 12, 17] in FC-MNIST and [5, 9, 17, 18] in FC-FASHION. The median values for sparsification (see Table 4) are [40, 27, 14, 46] in FC-MNIST and [43, 26, 30, 33] in FC-FASHION. In FC-MNIST and FC-FASHION, q_1 converges to low values while y_1 converges to high values. For example, q_1 converges to values between 4 and 14, y_1 converges to values between 34 and 50 in FC-MNIST, and FC-FASHION gets similar values. That indicates that high quantisation and sparsification can be performed while obtaining the best accuracies and minimising communication. The other q and y values are slightly more intermediate, compensating for the effort in the first layer (the most populated).

In C-MNIST and C-FASHION, the layers with the most influence on reducing communication are 3, 5, 7, and 9. The number of weights per connection between layers in the convolutional ANN is [800, 32, 25600, 32, 18432, 64, 36864, 64, 802816, 256, 2560, 10]. In this case, the median values for quantisation (see Table 5) are [19, 15, 14, 14, 14, 12, 17, 16, 6, 9, 12, 20] in C-MNIST and [13, 12, 18, 16, 15, 19, 15, 28, 12, 15, 2, 11] in C-FASHION. The median values for sparsification (see Table 5) are [19, 13, 35, 40, 35, 29, 39, 15, 46, 20, 21, 29] in C-MNIST and [18, 31, 22, 30, 17, 15, 13, 26, 14, 34, 38, 39] in C-FASHION. Of these, layer 9 is by far the most populous. In the case of C-MNIST, it is in layer 9 where the greatest quantisation and sparsification effort is made. However, in the case of C-FASHION, it is interesting to note that q_{11} converges to values between 1 and 2 (mostly to 2), which is an extremely high quantisation level. That is a striking result since layer 11 is one of the last layers and not one of the most populated. Moreover, the sparsification performed on y_{11} also reaches high values (median of 38). In the case of this dataset, the algorithm focused on this layer and managed to compress it extremely without damaging the accuracy. Moreover, in C-FASHION, y_9 remained low (median value of $y_9 = 14$), and q_9 did not perform extreme quantisation either (median value of $q_9 = 12$). For this dataset and ANN topology, the algorithm decided not to significantly reduce the most populated layer but focus on extremely compressing layer 11. These results contrast with the parameters obtained in the population-wide Pareto of C-FASHION, in which case, as we explained above, there are many solutions where higher sparsification was performed on y_9. The algorithm found two clear paths to get high-accuracy solutions. Although in the solutions of the pseudo-optimal Pareto (see Fig. 14), using a large sparsification in y_9 had good results, when looking at the solutions with better accuracy (see Fig. 16), this does not occur. That may be caused by using EDA to the solutions with better accuracy, allowing more exploration and escaping from local optima.

In the highest accuracies, the learning rate η converges to 0.0020 in FC-MNIST, uniformly between 0.0015 and 0.0020 in FC-FASHION, and 0.02 in both C-MNIST and C-FASHION. That is, they converge towards the highest allowed η values. As for the minibatch size b, it always converges to 4. The technique of local weight sharing or cumulative gradient sharing g generally converges to 0 (weight sharing), except in C-MNIST, where it converges to 1 (gradient sharing). The aggregation technique ϕ is uniformly between 0 (*equal*) and 1 (*proportional*) in FC-MNIST and FC-FASHION but converges to 0 in C-MNIST and to 1 in C-FASHION. The result for ϕ is different than expected, as one would assume that devices performing more local steps should be more important in the aggregation. However, the difference in device speeds (heterogeneity) may need to be more significant for this parameter to become more critical, which will have to be analysed in more detail in future work. The results show that this parameter becomes more significant as the number of layers and dataset complexity increase. Regarding the techniques used to deal with the zeros produced in sparsification o, it converges to 1 (*skip*) except in FC-MNIST where it converges to 0 (*ignore*). This technique seems to be more critical when using more layers, i.e., C-MNIST and C-FASHION.

In conclusion, these results have shown insights into effective strategies to reduce communication and achieve the best levels of accuracy when using these ANN topologies and datasets: (1) reduce the number of participating devices m in each round of communication, (2) generally perform a medium quantisation and sparsification in most of the layers and increase it in the layers with a higher number of weights, (3) increase the number of local steps E to high values before

Table 5
Median parameter values for quantisation and sparsification in the top 15 accuracies of FL-OPT Pareto front in C-MNIST and C-FASHION.

Number of weights	l_1	l_2	l_3	l_4	l_5	l_6	l_7	l_8	l_9	l_{10}	l_{11}	l_{12}
	800	32	25 600	32	18 432	64	36 864	64	**802 816**	256	2560	10
Quantisation 1...32	q_1	q_2	q_3	q_4	q_5	q_6	q_7	q_8	q_9	q_{10}	q_{11}	q_{12}
C-MNIST	19	15	14	14	14	12	17	16	**6**	9	12	20
C-FASHION	13	12	18	16	15	19	15	28	12	15	**2**	11
Sparsification 0...50	y_1	y_2	y_3	y_4	y_5	y_6	y_7	y_8	y_9	y_{10}	y_{11}	y_{12}
C-MNIST	19	13	35	40	35	29	39	15	**46**	20	21	29
C-FASHION	18	31	22	30	17	15	13	26	14	34	38	**39**

communicating. However, the E parameter is adjusted more often to compensate for the effort made by the other parameters in reducing communication. In the case of the highest accuracy solutions, the median of the parameter values for both datasets tends to converge to similar values, giving us a rough idea of how to adjust these parameters. In future work, we want to analyse further the correlation between the parameters the algorithm converges to across different datasets, see if it is possible to find connections and generalise these results to a greater extent. Also, the FL-OPT algorithm sometimes opts for extreme compression in less populated layers while decreasing compression in the most populated layer, an issue that requires further analysis in future work. As for the aggregation technique, it is more critical when the number of layers increases, and the dataset is more complex. In future work, it will also be necessary to investigate to what extent the aggregation technique affects when heterogeneity increases. As for the technique used to deal with zeros, the best strategy is to skip them when performing the aggregation.

5. Conclusions and future works

The Federated Learning Optimiser (FL-OPT) consisting of an NSGA-II coupled with an EDA for solving the Communication Reduction Problem (CRP) was proposed in this work. The experimental study simulated a server-client architecture with four devices of varying processing speeds. Different ANN model topologies were tested, including convolutional and fully connected ANNs. Two datasets, MNIST and Fashion-MNIST, were used for experimentation. A comparison has been made against three approaches: (I) vanilla NSGA-II, (II) fixed configuration with 100% communications, and (III) fixed configuration with 50% communications. The distribution and correlation between the resulting parameters were also analysed.

The FL-OPT method demonstrated empirically superior accuracy compared to other approaches, including the full-communication setting, while reducing communications by around 1,000 times in most cases and up to 10,000 times in some cases compared to the full-communication configuration (comm = 1). These results indicate that the noise introduced in the learning process by adjusting these parameters makes it easier to escape from local optima and to generalise better. FL-OPT also gets a higher hypervolume than other methods, exploring the Pareto front more efficiently, increasing the diversity of solutions, and finding more non-dominated solutions on the extremes of the Pareto front. Moreover, the resulting population of solutions assists in analysing correlations and distributions of the final parameters, aiding in anticipating outcomes when adjusting parameters. The proposed algorithm highly quantises and sparsifies layers with more weights for various datasets and ANN topologies while retaining slightly more intermediate values in the remaining layers.

Regarding limitations and future work, the NSGA-II is a well-established multi-objective EA effective for two or three objectives but may face challenges when exceeding three goals. Newer many-objective optimisation algorithms and more computationally efficient non-dominated sorting algorithms are advisable for dealing with larger population sizes or many objectives. However, in practical terms, the primary computational burden arises from extensive and time-consuming evaluations of objective functions, particularly in distributed model training, making it highly computationally intensive. To address this challenge, surrogate-assisted evolutionary or Bayesian optimisation methods can significantly reduce computational expenses [20].

In future work, we aim to analyse further the relationship between the parameters to which this algorithm converges for one dataset and a different dataset. We want to investigate whether it is possible to identify connections and generalise these results to a greater extent. Additionally, we aim to explore how the aggregation technique affects learning when heterogeneity increases. Moreover, investigate more deeply why the FL-OPT algorithm sometimes performs extreme compression on layers that are not the most populated while reducing the compression level on the most populated layer. Further, we plan to conduct experiments using more clients with unbalanced and *non-i.i.d.* datasets. We also plan to compare different aggregation methods, reduce computation using early-stopping, use real distributed edge devices, add more objectives (e.g., reducing processing, battery consumption, and more), and conduct research on optimising model compression before, during and after the training.

CRediT authorship contribution statement

José Ángel Morell: Writing – original draft, Visualization, Software, Methodology, Investigation, Conceptualization. **Zakaria Abdelmoiz Dahi:** Writing – review & editing, Visualization, Methodology, Investigation. **Francisco Chicano:** Writing – review & editing, Supervision, Methodology, Investigation, Funding acquisition. **Gabriel Luque:** Writing – review & editing, Supervision, Methodology, Investigation, Funding acquisition. **Enrique Alba:** Writing – review & editing, Supervision, Methodology, Investigation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

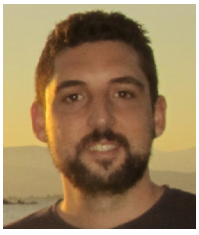
No data was used for the research described in the article.

Acknowledgements

This research is funded by PID 2020-116727RB-I00 (HUmove) funded by MCIN/AEI/ 10.13039/501100011033, Spain; and TAILOR ICT-48 Network (No 952215) funded by EU Horizon 2020 research and innovation programme, Spain. Funding for open access charge: Universidad de Málaga / CBUA. The authors thank the Supercomputing and Bioinnovation Center (SCBI) for their provision of computational resources and technical support. The views expressed are purely those of the writer and may not in any circumstances be regarded as stating an official position of the European Commission. The second author declares that this work was made and initially submitted while at the University of Malaga.

References

[1] I. Goodfellow, Y. Bengio, A. Courville, Deep learning, MIT Press, 2016.
[2] W. Shi, J. Cao, Q. Zhang, Y. Li, L. Xu, Edge computing: Vision and challenges, IEEE Internet Things J. 3 (5) (2016) 637–646.
[3] M. Chiang, T. Zhang, Fog and IoT: An overview of research opportunities, IEEE Internet Things J. 3 (6) (2016) 854–864.
[4] M. Finnegan, Boeing 787s to create half a terabyte of data per flight, says virgin atlantic, Comput.world UK 6 (2013).
[5] Y. Liu, J. Nie, X. Li, S.H. Ahmed, W.Y.B. Lim, C. Miao, Federated learning in the sky: Aerial-ground air quality sensing framework with UAV swarms, IEEE Internet Things J. 8 (12) (2020) 9827–9837.
[6] J. Posner, L. Tseng, M. Aloqaily, Y. Jararweh, Federated learning in vehicular networks: opportunities and solutions, IEEE Netw. 35 (2) (2021) 152–159.
[7] P. Kairouz, H.B. McMahan, B. Avent, A. Bellet, M. Bennis, A.N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, et al., Advances and open problems in federated learning, Found. Trends® Mach. Learn. 14 (1–2) (2021) 1–210.
[8] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Artificial Intelligence and Statistics, PMLR, 2017, pp. 1273–1282.
[9] A. Tak, S. Cherkaoui, Federated edge learning: Design issues and challenges, IEEE Netw. (2020).
[10] G. Long, Y. Tan, J. Jiang, C. Zhang, Federated learning for open banking, in: Federated Learning, Springer, 2020, pp. 240–254.
[11] B. Pfitzner, N. Steckhan, B. Arnrich, Federated learning in a medical context: a systematic literature review, ACM Trans. Internet Technol. (TOIT) 21 (2) (2021) 1–31.
[12] J. Xu, B.S. Glicksberg, C. Su, P. Walker, J. Bian, F. Wang, Federated learning for healthcare informatics, J. Healthc. Informat. Res. 5 (1) (2021) 1–19.
[13] D. Alistarh, D. Grubic, J.Z. Li, R. Tomioka, M. Vojnovic, QSGD: Communication-efficient SGD via gradient quantization and encoding, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS '17, 2017, pp. 1707–1718.
[14] J. Wangni, J. Wang, J. Liu, T. Zhang, Gradient sparsification for communication-efficient distributed optimization, in: Proceedings of the 32nd International Conference on Neural Information Processing Systems, NIPS '18, 2018, pp. 1306–1316.
[15] Z. Zhao, Y. Mao, Y. Liu, L. Song, Y. Ouyang, X. Chen, W. Ding, Towards efficient communications in federated learning: A contemporary survey, J. Franklin Inst. B (2023).
[16] M. Luo, F. Chen, D. Hu, Y. Zhang, J. Liang, J. Feng, No fear of heterogeneity: Classifier calibration for federated learning with non-iid data, Adv. Neural Inf. Process. Syst. 34 (2021) 5972–5984.
[17] J.Á. Morell, E. Alba, Dynamic and adaptive fault-tolerant asynchronous federated learning using volunteer edge devices, Future Gener. Comput. Syst. 133 (2022) 53–67.
[18] T. Nishio, R. Yonetani, Client selection for federated learning with heterogeneous resources in mobile edge, in: ICC 2019-2019 IEEE Int. Conference on Communications, ICC, IEEE, 2019, pp. 1–7.
[19] C. Xu, Y. Qu, Y. Xiang, L. Gao, Asynchronous federated learning on heterogeneous devices: A survey, 2021, arXiv preprint arXiv:2109.04269.
[20] H. Zhu, Y. Jin, Multi-objective evolutionary federated learning, IEEE Trans. Neural Netw. Learn. Syst. 31 (4) (2019) 1310–1322.
[21] M.W. Przewozniczek, M.M. Komarnicki, B. Frej, Direct linkage discovery with empirical linkage learning, in: Proceedings of the Genetic and Evolutionary Computation Conference, 2021, pp. 609–617.
[22] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Trans. Ev. Comp. 6 (2) (2002) 182–197.
[23] M. Pelikan, Probabilistic model-building genetic algorithms, in: Proceedings of the 13th Annual Conference Companion on Genetic and Evolutionary Computation, 2011, pp. 913–940.
[24] J.Á. Morell, Z.A. Dahi, F. Chicano, G. Luque, E. Alba, Optimising communication overhead in federated learning using NSGA-II, in: International Conference on the Applications of Evolutionary Computation (Part of EvoStar), Springer, 2022, pp. 317–333.
[25] H. Mühlenbein, G. Paass, From recombination of genes to the estimation of distributions I. binary parameters, in: International Conference on Parallel Problem Solving from Nature, Springer, 1996, pp. 178–187.
[26] R. Mayer, H.-A. Jacobsen, Scalable deep learning on distributed infrastructures: Challenges, techniques, and tools, ACM Comput. Surv. 53 (1) (2020) 1–37.
[27] Y. Zhou, Q. Ye, J. Lv, Communication-efficient federated learning with compensated overlap-fedavg, IEEE Trans. Parallel Distrib. Syst. 33 (1) (2021) 192–205.
[28] T. Ben-Nun, T. Hoefler, Demystifying parallel and distributed deep learning: An in-depth concurrency analysis, ACM Comput. Surv. 52 (4) (2019) 1–43.
[29] Y.J. Cho, J. Wang, G. Joshi, Client selection in federated learning: Convergence analysis and power-of-choice selection strategies, 2020, arXiv preprint arXiv:2010.01243.
[30] Y. Lu, X. Huang, K. Zhang, S. Maharjan, Y. Zhang, Blockchain empowered asynchronous federated learning for secure data sharing in internet of vehicles, IEEE Trans. Veh. Technol. 69 (4) (2020) 4298–4311.
[31] S. Niknam, H.S. Dhillon, J.H. Reed, Federated learning for wireless communications: Motivation, opportunities, and challenges, IEEE Commun. Mag. 58 (6) (2020) 46–51.
[32] Z. Yang, M. Chen, K.-K. Wong, H.V. Poor, S. Cui, Federated learning for 6G: Applications, challenges, and opportunities, Engineering 8 (2022) 33–41.
[33] Y. Chen, X. Sun, Y. Jin, Communication-efficient federated deep learning with layerwise asynchronous model update and temporally weighted aggregation, IEEE Trans. Neural Netw. Learn. Syst. 31 (10) (2019) 4229–4238.
[34] Z. Chen, W. Liao, K. Hua, C. Lu, W. Yu, Towards asynchronous federated learning for heterogeneous edge-powered internet of things, Digit. Commun. Netw. 7 (3) (2021) 317–326.
[35] N.S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, P.T.P. Tang, On large-batch training for deep learning: Generalization gap and sharp minima, 2016, arXiv preprint arXiv:1609.04836.
[36] M. López-Ibáñez, J. Dubois-Lacoste, L.P. Cáceres, M. Birattari, T. Stützle, The irace package: Iterated racing for automatic algorithm configuration, Oper. Res. Perspect. 3 (2016) 43–58.
[37] H. Ishibuchi, N. Tsukamoto, Y. Sakane, Y. Nojima, Indicator-based evolutionary algorithm with hypervolume approximation by achievement scalarizing functions, in: Proceedings of the 12th Annual Conference on Genetic and Evolutionary Computation, 2010, pp. 527–534.
[38] E. Zitzler, S. Künzli, Indicator-based selection in multiobjective search, in: International Conference on Parallel Problem Solving from Nature, Springer, 2004, pp. 832–842.
[39] L. Myers, M.J. Sirois, Spearman correlation coefficients, differences between, Encycl. Statist. Sci. 12 (2004).



José Ángel Morell received a B.Eng. degree (Hons.) in computer science from the Universidad Nacional de Educación a Distancia (UNED) in 2016, an M.Sc. degree (Hons.) in software engineering and artificial intelligence from the University of Málaga in 2017, and a Ph.D. degree in computer science from the University of Málaga in 2023. His research interests include the design of optimisation and machine learning distributed algorithms adapted to edge computing.

Zakaria Abdelmoiz Dahi is a researcher at the French National Institute of Research in Informatics and Automation (INRIA) of the University of Lille (France). His research interests include the use and design of classical and quantum optimisation and machine-learning techniques for solving real-world problems.

Francisco Chicano received the degree in physics from the National Distance Education University and the Ph.D. degree in computer science from the University of Málaga. Since 2008, he has been with the Department of Languages and Computing Sciences, University of Málaga. His research interests include the application of search techniques to software engineering problems and the use of theoretical results to efficiently solve combinatorial optimisation problems. He has also been the program chair and the editor-in-chief in international events. He is in the Editorial Board of Evolutionary Computation journal, Engineering Applications of Artificial Intelligence, Journal of Systems and Software, ACM Transactions on Evolutionary Learning and Optimisation, and Mathematical Problems in Engineering.

Gabriel Luque received the Ph.D. degree in computer science from the Universidad de Málaga, Spain, in 2006, where he is currently an Associate Professor with the Department of Languages and, Computer Science. He works with varied teaching duties: networks and distributed systems, programming, and also software engineering, both at graduate and master/doctoral programs. He is also currently working on the design of theory-driven algorithms and on the

application of parallel technique to solve dynamic optimisation problems. He coauthored more than 90 international publications and coauthored a book Parallel Genetic Algorithms. He has also participated organising and chairing several special sessions and workshops about metaheuristics, parallel techniques, and tools for the development of optimisation methods. He has directed and participated in several research projects and some projects for innovation in companies (AOP, VATIA, INDRA, EMERGIA, SECMOTIC, or ArcerlorMittal). His H index is 20, with more than 2,000 cites to his work. His current research interests include the design of new metaheuristics, specifically on parallel algorithms, and their application to complex problems in the domains of bioinformatics, smart cities, networking, and combinatorial optimisation in general.

Enrique Alba had his degree in engineering and PhD in Computer Science in 1992 and 1999, respectively, by the University of Málaga (Spain). He works as a Full Professor in this university with varied teaching duties: data communications, distributed programing, software quality, and also evolutionary algorithms, bases for R+D+i and smart cities, both at graduate and master/doctoral programs. Prof. Alba leads an international team of researchers in the field of complex optimisation/learning with applications in smart cities, bioinformatics, software engineering, telecoms, and others. In addition to the organisation of international events (ACM GECCO, IEEE IPDPS-NIDISC, IEEE MSWiM, IEEE DS-RT, smart-CT...) Prof. Alba has offered dozens postgraduate courses, more than 70 seminars in international institutions, and has directed many research projects (9 with national funds, 7 in Europe, and numerous bilateral actions). Also, Prof. Alba has directed 12 projects for innovation in companies (OPTIMI, Tartessos, ACERINOX, ARELANCE, TUO, INDRA, AOP, VATIA, EMERGIA, SECMOTIC, ArcelorMittal, ACTECO, CETEM, EUROSOTERRADOS) and has worked as invited professor at INRIA, Luxembourg, Ostrava, Japan, Argentina, Cuba, Uruguay, and Mexico. He is editor in several international journals and book series of Springer-Verlag and Wiley, as well as he often reviews articles for more than 30 impact journals. He is included in the list of most prolific DBLP authors, and has published 130 articles in journals indexed by ISI, 11 books, and hundreds of communications to scientific conferences. He is included in the top ten most relevant researchers in Informatics in Spain (fifth position in ISI), and is the most influent researcher of UMA in engineering (webometrics), with 14 awards to his professional activities. Pr. Alba's H index is 65, with more than 19,000 cites to his work.